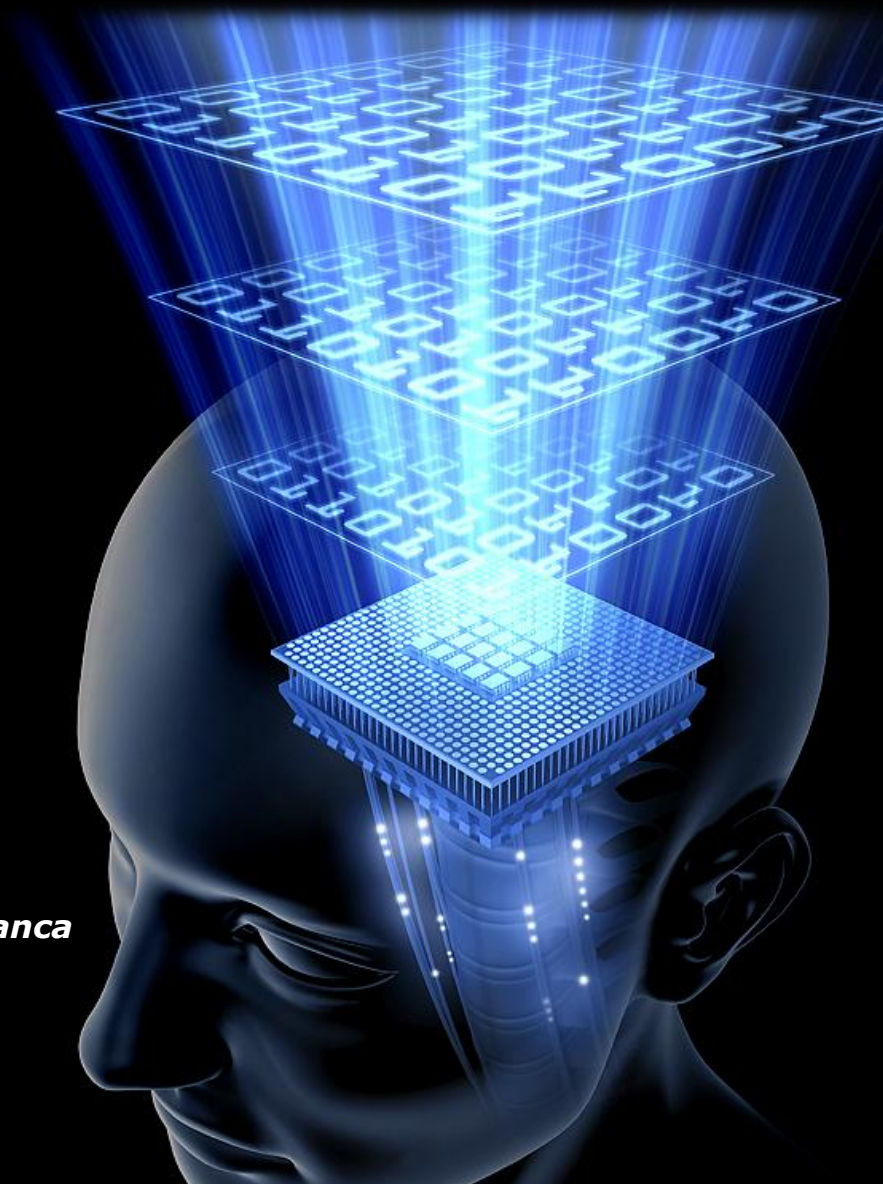
Reconocimiento de dígitos con redes neuronales en Vitis HLS

Arquitecturas Especializadas UMA

Este proyecto implementa un sistema de reconocimiento de dígitos manuscritos utilizando redes neuronales en la herramienta Vitis HLS. El objetivo es diseñar e implementar un modelo neuronal optimizado para hardware, capaz de clasificar imágenes de dígitos (por ejemplo, del conjunto de datos MNIST).

El modelo se describe en C/C++ y se sintetiza en lógica hardware para su posterior implementación en FPGA. El código está optimizado mediante técnicas de paralelismo, pipeline y representación en punto fijo, buscando un equilibrio entre precisión, velocidad y uso de recursos.



Realizado por:

- ***José Antonio Casado Molina***
- ***Diego Alejandro Machaca Huaranca***
- ***Claudio María Martínez Velasco***

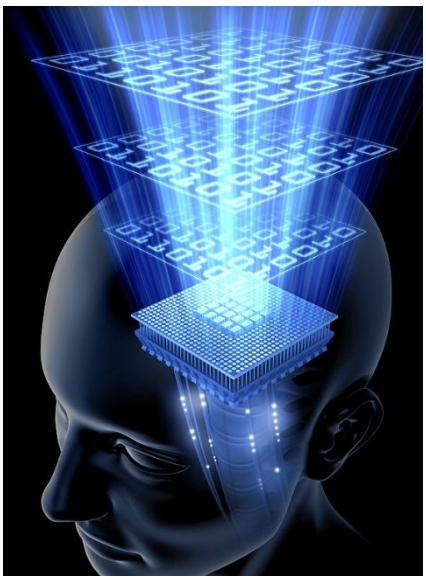
Índice

Apéndice 3
Redes Neuronales 3
1. Capa de entrada 3
2. Capas ocultas 3
3. Capa de salida 3
4. Entrenamiento 4
Proceso de entrenamiento y ajuste de pesos 4
Interpretación del funcionamiento de una red funcional 5
Tipos de Redes Neuronales 6
Redes neuronales feedforward (FFNN) 6
Redes neuronales recurrentes (RNN) 6
Redes neuronales convolucionales (CNN) 6
Bibliotecas y herramientas compatibles con Vitis HLS para redes neuronales 7
hls4ml 7
Vitis Vision Library 7
FINN 7
Introducción 8
Arquitectura Base de la OCR 8
Explicación del código 9
Análisis de Resultados 12
Arquitectura Iterativa de la OCR 13
Explicación del código 13
Análisis de Resultados 14
Arquitectura Rápida de la OCR 15
Explicación del código 15
Análisis de Resultados 18
Conclusiones 19
Anexo 20
Explicación del código 20
Flujo de datos para la CNN acelerada 21
Salida tras ejecutar el código de testeo 22
Bibliografía 23

Apéndice

En esta sección del documento, se **incluye información adicional** que no es esencial para el cuerpo principal, pero que puede resultar útil al lector. Por ejemplo, el funcionamiento de una Red Neuronal, sus tipos y las herramientas necesarias para trabajar con redes neuronales en la plataforma usada para este proyecto, en este caso Vitis HLS.

Redes Neuronales



Una red neuronal es un programa o modelo de *machine learning*[1] que toma decisiones de forma similar al cerebro humano, al emplear procesos que imitan la forma en que las neuronas biológicas trabajan juntas para identificar fenómenos, sopesar opciones y llegar a conclusiones. Se compone de **neuronas** organizadas en **capas**: una capa de entrada, una o varias capas ocultas, y una capa de salida.

1. **Capa de entrada**: Recibe los datos del problema. Por ejemplo, si se trata de una imagen de un dígito de 28x28 píxeles, se tendrán 784 entradas (una por píxel).

2. **Capas ocultas**: Cada neurona en estas capas toma las salidas de la capa anterior, las multiplica por ciertos **pesos**, les suma un **sesgo** y aplica una **función de activación** (como ReLU o sigmoide). Esto permite a la red aprender patrones complejos.

Fórmula básica de una neurona:

$$\boldsymbol{salida} = \boldsymbol{f}\left(\sum_{\boldsymbol{i}} \boldsymbol{w_i} \cdot \boldsymbol{x_i} + \boldsymbol{b}\right)$$

donde:
$\boldsymbol{x_i}$: entradas,
$\boldsymbol{w_i}$: pesos,
$\boldsymbol{b}$: sesgo,
$\boldsymbol{f}$: función de activación.

3. **Capa de salida**: Genera la predicción final. En un clasificador de dígitos (0-9), hay 10 neuronas de salida, y la red escoge la que tenga la mayor activación como resultado.

4. **Entrenamiento**: Se ajustan los pesos y sesgos mediante un proceso llamado **retropropagación** usando un algoritmo como **descenso del gradiente**, comparando la salida obtenida con la salida esperada.

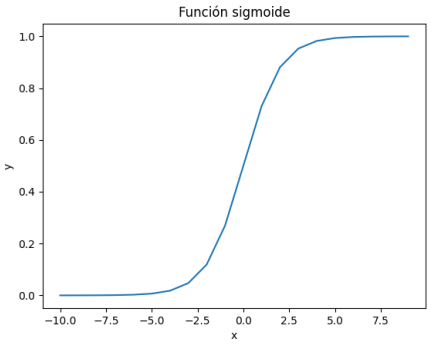
Proceso de entrenamiento y ajuste de pesos

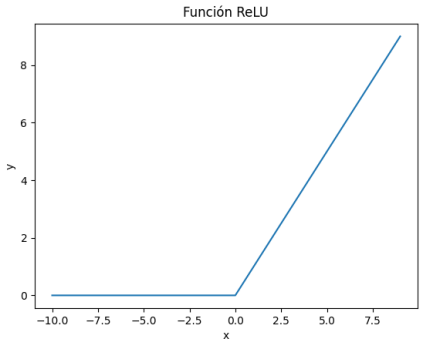
El entrenamiento de una red neuronal consiste en alimentarla con datos de entrada acompañados de una salida esperada. Si la respuesta generada por la red no coincide con la esperada, se modifican los pesos de las conexiones internas para mejorar su rendimiento. Este ciclo se repite múltiples veces hasta que la red es capaz de producir respuestas correctas de forma consistente.

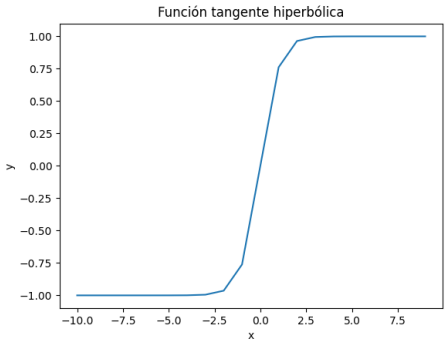
Existen diferentes métodos para entrenar redes neuronales, y uno de los más utilizados es el algoritmo de retropropagación. Este método calcula la diferencia entre la salida real de la red y la salida deseada, y con base en ese error, ajusta los pesos de las conexiones con el objetivo de reducir progresivamente dicho error.

Las funciones de activación desempeñan un papel esencial en el funcionamiento de las redes neuronales. Cada neurona aplica una de estas funciones para decidir qué salida generar a partir de los valores que recibe como entrada.
Estas funciones pueden clasificarse como lineales o no lineales. Las funciones lineales realizan operaciones simples, como multiplicar la entrada por un peso y sumar un sesgo. En cambio, las funciones no lineales introducen una mayor complejidad, lo que permite que la red neuronal capture patrones y relaciones más sofisticadas en los datos.

Elegir una función de activación adecuada es crucial, ya que puede influir directamente en la capacidad de la red para aprender correctamente y adaptarse a datos nuevos. Algunas de las funciones más utilizadas en la práctica son la sigmoide, la ReLU (Unidad Lineal Rectificada) y la tangente hiperbólica.

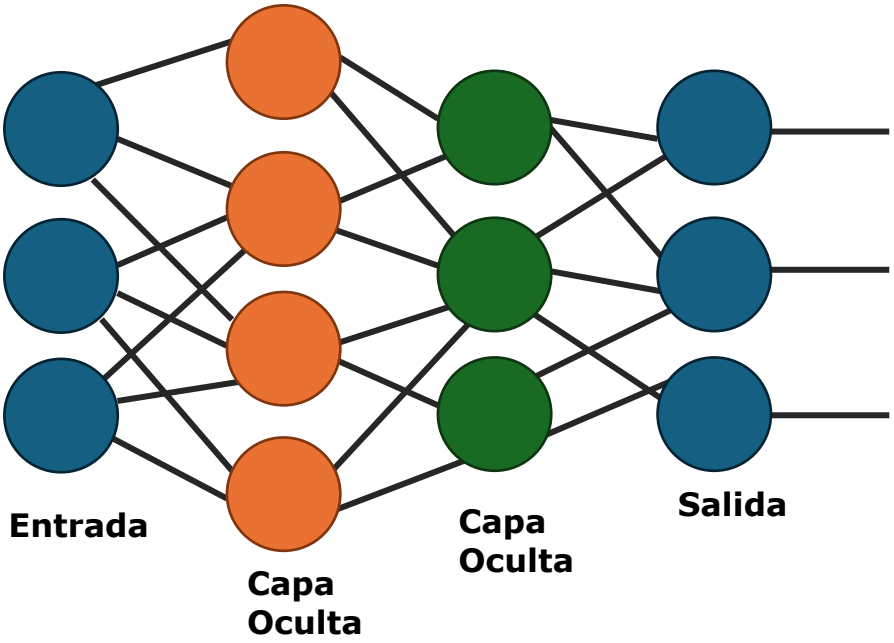






Interpretación del funcionamiento de una red funcional

Una red neuronal se compone principalmente de varias capas de neuronas conectadas entre sí. La primera capa, llamada capa de entrada, recibe la información inicial y la transmite a través de la red. La última capa, conocida como capa de salida, genera el resultado final. Entre ambas se encuentran las capas ocultas, que se encargan de transformar y analizar los datos recibidos.



Cada neurona está enlazada con otras mediante conexiones llamadas sinapsis, similares a las del cerebro humano. Estas conexiones tienen un peso que define cuán influyente es la señal transmitida. Durante el entrenamiento de la red, estos pesos se modifican mediante algoritmos de aprendizaje, con el objetivo de mejorar la precisión de las predicciones o respuestas generadas.

Tipos de Redes Neuronales

Distintas redes neuronales tienen distintas aplicaciones, dependiendo del uso que sea necesario darles para el tratamiento de sus datos, las más importantes para los actuales usos de estas son las llamadas feedforward, recurrentes o convolucionales.

Redes neuronales feedforward (FFNN)

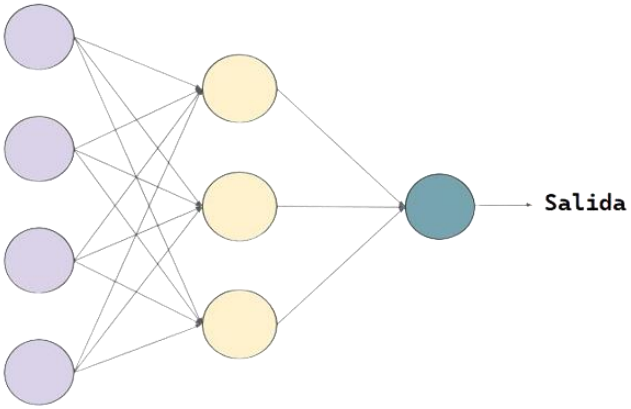
Son las redes neuronales más simples. La información fluye en una sola dirección, desde la capa de entrada hacia la salida, pasando por capas ocultas. No tienen ciclos ni retroalimentación. Son ideales para tareas como clasificación o regresión con datos estructurados.

Redes neuronales recurrentes (RNN)

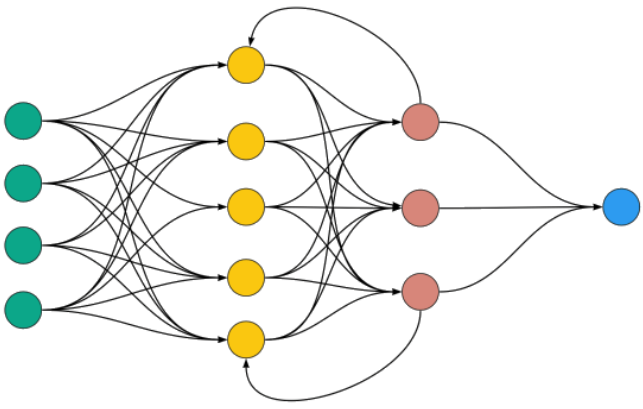
Están diseñadas para procesar secuencias de datos. A diferencia de las *feedforward*, las RNN tienen conexiones que forman bucles, lo que les permite "recordar" información previa. Se usan comúnmente en procesamiento de lenguaje natural, reconocimiento de voz y series temporales.

Redes neuronales convolucionales (CNN)

Son especializadas en el procesamiento de datos con estructura espacial, como imágenes. Utilizan capas convolucionales que detectan patrones locales (como bordes o texturas). Se aplican ampliamente en visión por computadora, detección de objetos y clasificación de imágenes.



Red Neuronal feedforward (FFNN)



Red Neuronal recurrente (RNN)

Bibliotecas y herramientas compatibles con Vitis HLS para redes neuronales

hls4ml: Es una herramienta que permite convertir modelos de redes neuronales entrenados en frameworks como Keras[2] o PyTorch[3] en código C++ optimizado para síntesis en Vitis HLS.
Facilita la implementación de modelos preentrenados en hardware FPGA. Soporta capas comunes como convoluciones, activaciones (ReLU, sigmoid), pooling, etc.



Vitis Vision Library: Es una biblioteca de funciones de procesamiento de imágenes optimizadas para Vitis HLS.
Incluye funciones para preprocesamiento de imágenes, como redimensionamiento, filtrado, etc. Facilita la integración de procesamiento de imágenes en tu flujo de diseño.



FINN: Es un framework de Xilinx para implementar redes neuronales cuantizadas en FPGAs.
Especializado en redes con baja precisión (por ejemplo, 1-2 bits), lo que reduce el uso de recursos y aumenta la velocidad. Incluye herramientas para entrenamiento consciente de la cuantización y generación de aceleradores personalizados.

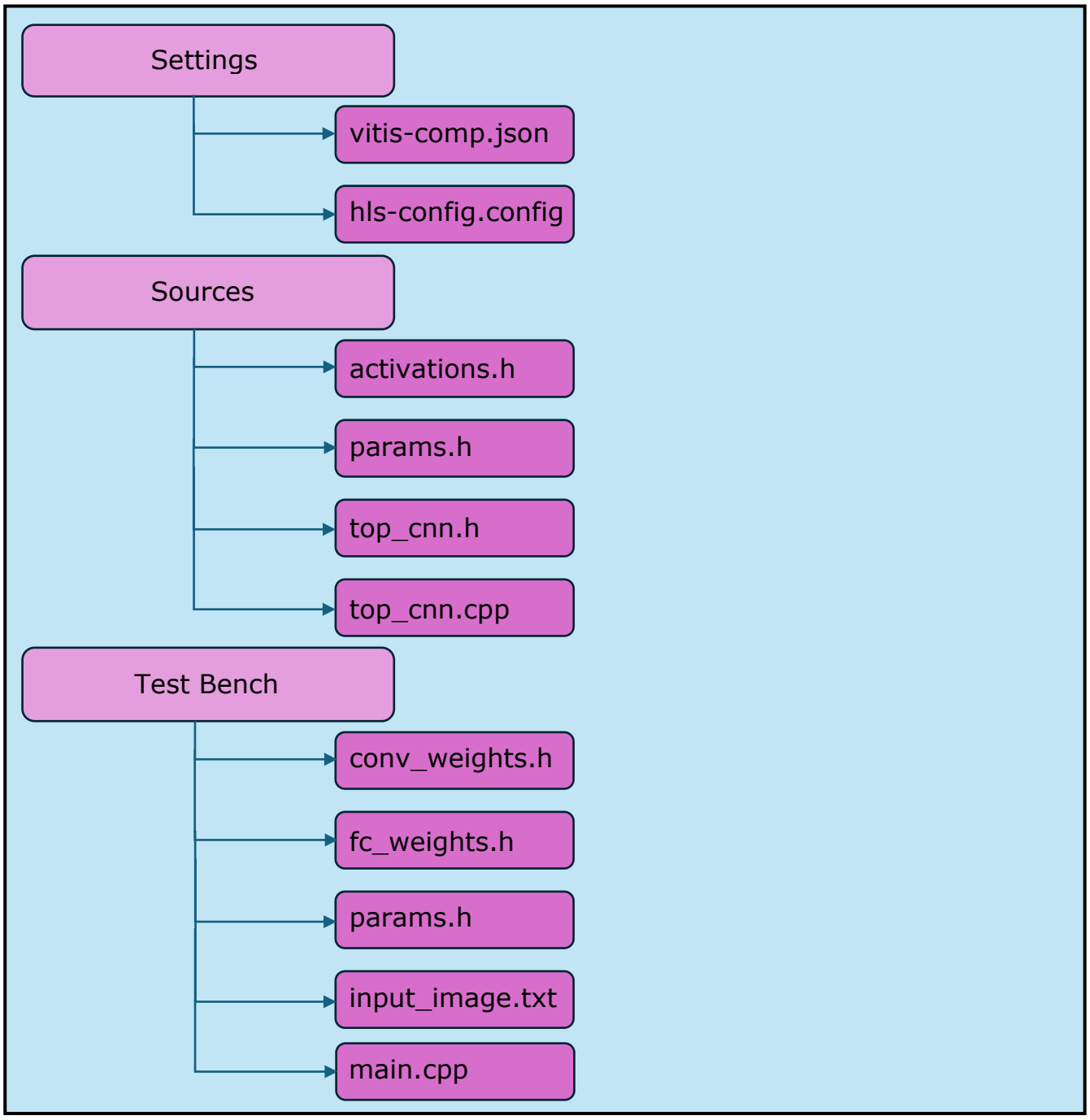
Introducción

A continuación, se mostrarán distintas arquitecturas, en base a los parámetros resultantes de las síntesis ejecutadas en el programa Vitis HLS. Para conocer más acerca de cómo funciona, por favor visite el siguiente enlace [4].

Arquitectura Base de la OCR

Caso de uso: *Versión preliminar y no optimizada de un detector de dígitos utilizando una red neuronal.*

En esta primera versión, hemos creado un componente HLS cuya jerarquía de archivos es la siguiente:



A continuación, se explica resumidamente la función de cada archivo en el componente:

- *hls-config.config* → archivo de configuración, en él se especifica el dispositivo FPGA, se indica que la función **principal (top-level)** y se incluye todos los archivos necesarios para la síntesis.

- *activations.h* → define **funciones de activación típicas de redes neuronales.**

- *params.h* → se define principalmente **dimensiones de tensores**, **tamaños de pesos y activaciones**, y algunos **hiperparámetros de red**.

- *top_cnn.h* → se definen las funciones principales para importarlas en otros archivos del componente.

- *top_cnn.cpp* → en él se encuentra toda la lógica del proyecto. Se integran las capas fundamentales de una CNN: **convolución, pooling y fully connected (FC)**, y contiene la función top_cnn(...), que actúa como **punto de entrada (top-level)** para el módulo sintetizable.

- *conv_weights.h* → define e inicializa los pesos, los bias y la imagen de entrada de una red neuronal convolucional (CNN).

- *fc_weights.h* → define e inicializa los **parámetros de la capa FC.**

- *main.cpp* → archivo de testbench, ejecuta una red neuronal convolucional (CNN) embebida, leyendo una imagen desde un archivo de texto, pasándola por la red (top_cnn), e imprimiendo la clase predicha con sus probabilidades.

Explicación del código

El archivo *top_cnn.cpp* contiene la lógica de la síntesis del código, es decir, las funciones que se optimizarán mediante pragmas y que ejecutan las funcionalidades de la red neuronal. Este código se divide en 4 funciones: **fc**, **pool**, **conv** y **top_cnn**.

1. **Función Full Connected (FC)**

```
void fc (float out[], float in[], float weight[FC_WEIGHTS_H][FC_WEIGHTS_W],
float bias[FC_BIAS_SIZE])
```

En esta función se implementa una **capa fully connected** en una red neuronal, y aplica la función *softmax* al final para convertir las salidas en probabilidades.

Como parámetros de entrada tenemos:

- `float out[]`: vector de salida (tendrá las probabilidades finales después del softmax).
- `float in[]`: vector de entrada (por ejemplo, salida de la capa de *pooling* o *flatten*).
- `float weight[][]`: matriz de pesos de la capa fully connected.
- `float bias[][]`: vector de bias.

En primer lugar, se crean 2 **vectores auxiliares**, uno de lectura (*read[]*) y otro de salida (*output[]*).
Mediante 2 bucles *for* recorremos la matriz de pesos *weight[][]*. Este bucle hace una **multiplicación matriz-vector**, que introduce al vector *output[]***.**
Acto seguido se añade el vector de *bias[]* más el vector auxiliar al vector de salida.
Por último, se aplica la función *softmax*, definida en la librería *activations.h*, que recibe como parámetros de entrada un puntero al vector de salida y el tamaño del vector auxiliar y convierte las salidas en **probabilidades normalizadas.**

2. Función Pool (pool)

```
void pool (float out[], float in[])
```

Esta función implementa una **operación de max pooling 2D** sobre una entrada multicanal, reduciendo su resolución espacial. En dicha operación:

- Se divide la entrada en bloques (por ejemplo 2×2).
- Para cada bloque se toma el **valor máximo**.
- Se salta a la siguiente ventana (según el *stride* o paso).

Como parámetros de entrada tenemos:

- `out[]`: vector de salida (después del max pooling), de tamaño reducido.
- `in[]`: vector de entrada (por ejemplo, salida de una capa convolucional) de tamaño *P_SIZE* × *P_SIZE* × *P_CHANNELS*.

En primer lugar, se crea un **buffer temporal** (*pool_buff[][]*) que guarda los máximos de cada región de pooling por canal.
En el bucle principal de la función se recorre **cada región 2×2 de cada canal**. Para cada elemento:

1. **Lee** el valor de entrada.
2. Si es la primera posición del **kernel**, lo guarda como inicial.
3. Si no, actualiza el valor máximo eligiendo el valor máximo entre *pool_buff[][]* y *read.*
4. Cuando llega al último elemento del kernel, guarda el **valor máximo** calculado en out.

Por último, el último bucle avanza el índice z para **saltar líneas** no cubiertas por el ***stride*** o desplazamiento. Es una forma de asegurarse que las regiones no se solapan.

3. Función Convolución (conv)

```
void conv (float out[], float in[], float
weight[CONV_KERNEL_SIZE][CONV_KERNEL_SIZE][CONV_CHANNELS][CONV_FILTERS],
float bias[CONV_BIAS_SIZE])
```

Esta función implementa **una operación de convolución 2D** para una imagen con múltiples canales (como una imagen RGB con 3 canales), aplicada con múltiples **filtros** (*kernels*), seguida por una función de activación *ReLU*.

Como parámetros de entrada tenemos:

- `out[]` : vector donde se almacenará la **salida de la convolución**.
- `in[]` : vector con la **imagen de entrada** (con todos sus canales).
- `weight[][][][]` : vector con los **pesos de los filtros** (*kernels*) de la convolución, 4 dimensiones:
  - alto del kernel,
  - ancho del kernel,
  - número de canales de entrada,
  - número de filtros (salida).
- `bias[]` : vector con los **sesgos** (*bias*) de cada filtro.

En primer lugar, se crea un **buffer temporal** (*conv_buff[]*) que guarda los máximos de cada región de pooling por canal.
Después, se **copia la entrada** *in[]* al buffer temporal a través del primer bucle.
Luego, para **cada posición** (i, j) donde el kernel puede "encajar" dentro de la imagen, moviéndose con un paso (*stride*) *CONV_STRIDE*, y para cada filtro disponible:

- Calcula la suma ponderada (**sum**) de la región de la imagen correspondiente al kernel, multiplicada por los pesos del kernel:
  - Se recorre el kernel en dos dimensiones (*row_offset*, *col_offset*).
  - Para cada canal de la imagen (*channel_offset*).
  - Se multiplica el valor de la imagen en la posición correspondiente por el peso del kernel en esa posición.
  - Se acumula esa suma en **sum**.
- Al resultado **sum** se le añade el sesgo correspondiente al filtro (*bias[filter]*).

Se aplica la función **ReLU** (*Rectified Linear Unit*), definida en la librería
Por último, se guarda ese resultado en la salida *out[]*, que tendrá la activación (**resultado**) para todos los filtros, en todas las posiciones posibles.

Análisis de Resultados

Tras realizar la **Síntesis C** que Vitis HLS nos ofrece, observamos los resultados de rendimiento y uso de recursos que también nos proporciona. En concreto, estudiaremos a fondo la ***latencia*** (en *ciclos de reloj*), el ***intervalo de iniciación***, el ***throughput*** y el uso de ***BRAM***, ***DSP***, ***LUT*** y ***FF***.

	Latencia (ciclos)	Interval	Throughput*	BRAM	DSP	FF	LUT
top_cnn	359.116	359.117	2808	65	91	28.585	50.322
bucle_fc11	54.157	54.081	18.464	0	0	1300	2746
bucle_fc2	154	11	6.493.506	0	0	2057	5094
bucle_pool14	21.634	21.633	46.223	1	0	242	1002
bucle_conv1	787	785	1.270.648	0	0	1031	122
bucle_conv2	280.567	280.567	3564	0	0	12.024	21.787

Como podemos observar, top_cnn sería la función principal, mientras que bucle_fc11 y bucle_fc2 pertenecen a la función fc(), *bucle_pool1* pertenece a la función pool() y bucle_conv1 y bucle_conv2 a la función conv().

Analizando los resultados obtenidos tras la síntesis podemos deducir que este programa tarda demasiados ciclos de reloj en producir una salida después de recibir una entrada (latencia), unos ~360.000 ciclos. Entorno al 70% de esta latencia la consume bucle_conv2 puesto que está formada por 6 bucles for internos y realiza una operación de suma compleja compuesta a su vez de diversas multiplicaciones. Se observa también que es este bucle el que más recursos de memoria utiliza, con respecto a los demás bucles, casi el 50% de la función principal. No utiliza ni BRAM ni DSP, pero sí LUT y FF para implementar funciones lógicas combinacionales y almacenar bits de información.

Podemos deducir, por tanto, que la *función de convolución* es la que más recursos necesita y la que más latencia produce, seguida de *la función full connected* y la *función de pooling*.

Por último, indicar, que, aunque no se refleje en la tabla anterior, Vitis HLS incluye paralelización (pipeline) a los bucles por defecto, incrementando el número de recursos de memoria necesarios. A continuación, se estudiará una arquitectura totalmente iterativa, que minimiza el uso de estos recursos, especialmente útil para FPGAs con poco tamaño de memoria.

Arquitectura Iterativa de la OCR

Caso de uso: *OCR para FPGAs de pocos recursos.*

Para esta arquitectura iterativa buscaremos minimizar al máximo el uso de recursos de memoria (*BRAM*, *DSP*, *LUT* y *FF*) para poder incluir nuestra OCR en una FPGA con pocos recursos, por ejemplo, las **xa7z** de la familia **XA Zynq-7000**.

La jerarquía de archivos para esta versión es la misma que para la Arquitectura Base, a diferencia del archivo top_cnn.cpp al cual le incluiremos algunas sentencias para conseguir nuestro objetivo.

Explicación del código

El código funciona de la misma manera que para la Arquitectura Base, pero, en esta ocasión aplicaremos pragmas en los bucles de las funciones, en particular, los pragmas **PIPELINE OFF** y **UNROLL OFF**, para evitar que Vitis HLS aplique estos recursos por defecto, mejorando la latencia, pero aumento el uso de recursos (lo contrario a lo que queremos).

1. **Función Full Connected (FC)**

Aplicaremos `#pragma PIPELINE OFF` en *bucle_fc1*, *bucle_fc11*y *bucle_fc2*.
También aplicaremos `#pragma UNROLL OFF` en *bucle_fc11*, el cual es un bucle interno con límite *FC_WEIGHTS_W* (posiblemente grande, pero para asegurar iteratividad). Evitamos así que se desenrolle y mejore la latencia.

2. **Función Pool (pool)**

Aplicamos `#pragma PIPELINE OFF` en *bucle_pool11* y *bucle_pool14*.
También aplicamos `#pragma UNROLL OFF` en *bucle_pool13*, ya que *P_KERNEL_SIZE* es un tamaño pequeño, y HLS podría desenrollarlo automáticamente y en *bucle_pool14*, por la misma razón.

3. **Función Convolución (conv)**

Se aplica `#pragma PIPELINE OFF` en *bucle_conv1* y en *bucle_conv24*.
También se aplica `#pragma UNROLL OFF` en *bucle_conv23* y en *bucle_conv25*.
En este último, se evita desenrollar operaciones por canal (crítico si *CONV_CHANNELS* es pequeño).

4. **Función principal (top)**

En la función principal (*top_cnn*) aplicamos `#pragma PIPELINE OFF` para eliminar también la paralelización en las llamadas a las funciones.

5. Función softmax

Está función está definida en *activations.h* y se aplica #pragma PIPELINE OFF en cada uno de los tres bucles. Esto se hace ya que Vitis también lo aplica por defecto al ser llamada desde otras funciones.

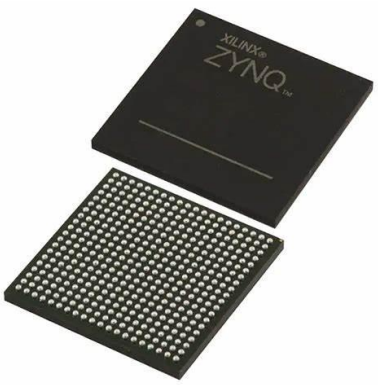
Análisis de Resultados

Al realizar la síntesis C de Vitis, analizamos los resultados en la siguiente tabla:

	Latencia (ciclos)	Interval	Throughput	BRAM	DSP	FF	LUT
top_cnn	21.096.441	21.096.442	47	65	91	18.169	30.670
bucle_fc1	4.234.464	783	236	-	-	-	-
bucle_fc2	770	-	1.298.701	-	-	-	-
bucle_pool1	45.370	-	22.040	-	-	-	-
bucle_conv1	3136	784	318.877	-	-	-	-
bucle_conv2	16.809.468	646.518	59	-	-	-	-

Como podemos observar, **se cumplen** las estimaciones del principio, aumentando muchísimo la latencia y el intervalo de iniciación, y disminuyendo el uso de recursos (FF y LUT) puesto que se realizan menos operaciones y se guardan menos bits de información, al no tener pragmas.

Nuestro programa cumple con las especificaciones de tiempo (reloj de **12 ns** e incertidumbre de 2ns) pero, según las características de la placa *XA7Z010-1CLG225I* (la que menos recursos tiene), **no** podríamos introducirla puesto que sobrepasa el uso de recursos de memoria. En este caso, la placa *xa7z020clg400-1I* sería la elegida para nuestra FPGA de bajo coste, puesto que **admite los resultados obtenidos**. Esta placa tiene un coste alrededor de *50$*, lo cual es perfecto para nuestra FPGA de pocos recursos y bajo coste.



xa7z020clg400-1I part

Arquitectura Rápida de la OCR

Caso de uso: *OCR para FPGAs que trabajen a baja latencia.*

Para esta arquitectura, el objetivo será disminuir la latencia con respecto a la Arquitectura Base, para conseguir un sistema de Reconocimiento numérico rápido. Para ello, debemos insertar pragmas al código. En esta ocasión se eligen los pragmas de pipeline, unroll y array_partition, explicados a continuación.

Explicación del código

En primer lugar, recordemos los pragmas que se van a utilizar:

- ***PIPELINE*** → permite la ejecución superpuesta de iteraciones consecutivas de un bucle, es decir, una nueva iteración puede comenzar antes de que la anterior haya terminado. El compilador intenta reducir el intervalo de iniciación (II), que es el número de ciclos de reloj entre el inicio de dos iteraciones consecutivas.

- ***UNROLL*** → desenrolla bucles de manera que se creen muchas operaciones independientes en lugar de un único conjunto de operaciones. El parámetro factor indica el número de veces que se desenrollará el bucle (si no se especifica, se desenrollará totalmente).

- ***ARRAY PARTITION*** → divide los arrays en diferentes bloques o bancos de memoria, de manera que se puede acceder a ellos de manera paralela. Cuenta con 3 tipos de partición:
    - **Complete** → divide cada uno de los elementos del array en un registro, de manera que se puede acceder en paralelo a todos los datos de un array.
    - **Block** → divide el array en bloques de valores consecutivos, almacenando cada uno de los bloques en memorias diferentes, pudiendo acceder a los elementos de diferentes bloques en paralelo.
    - **Cyclic** → divide en bloques con los valores del array original entrelazados.

- `void fc (float out[], float in[], float weight[FC_WEIGHTS_H][FC_WEIGHTS_W], float bias[FC_BIAS_SIZE])`

Para la función de full connected, aplicamos en primer lugar, el pragma ARRAY_PARTITION, para dividir los parámetros que recibe la función.

```
#pragma HLS ARRAY_PARTITION variable =in cyclic factor= 10 dim= 1
#pragma HLS ARRAY_PARTITION variable =out cyclic factor= 10 dim= 1
#pragma HLS ARRAY_PARTITION variable =weight cyclic factor= 10 dim= 2
#pragma HLS ARRAY_PARTITION variable =output complete
```

El pragma se configura para los arrays in, out, weight y output (creado posteriormente), para un factor de 10 y completo para el último. Esto se debe a que la variable *FC_ACT_SIZE*, que define el tamaño del array de salida output[], el cual se rellena, en parte, con el array in[] marca el factor a aplicar a dichos arrays y, para el caso del array weight[], lo hace la macro *FC_WEIGHTS_W*.

Dentro de los bucles aplicamos el pragma PIPELINE, para paralelizar cada iteración, con intervalo de iniciación 1.

```
#pragma HLS PIPELINE II= 1
```

Por último, dentro del bucle interno *bucle_fc11* se aplica el pragma UNROLL, para desenrollarlo, ya que el número de iteraciones es bajo.

```
#pragma HLS UNROLL factor= 4
```

Hay que tener en cuenta que, si se usara unroll completo (factor=10), se necesitarían más recursos hardware.

- `void pool (float out[], float in[])`

Para esta función aplicamos los mismos pragmas que anteriormente, justificando su uso de la misma manera.

```
#pragma HLS ARRAY_PARTITION variable =in cyclic factor= 16 dim= 1
#pragma HLS ARRAY_PARTITION variable =out cyclic factor= 16 dim= 1
```

Los arrays in[] y out[] tiene tamaño *P_SIZE* * *P_CHANNELS* (13 × 32 = 416). Particionar en 16 permite acceder a 16 elementos en paralelo, lo cual es más que suficiente para nuestra arquitectura.

```
#pragma HLS ARRAY_PARTITION variable =pool_buff complete dim= 2
```

Particiona completamente la segunda dimensión de pool_buff[], que es de tamaño 1, asegurando que cada elemento de la primera dimensión sea accesible en paralelo, eliminando cualquier posible cuello de botella por acceso a la segunda dimensión.

Por último, se aplica el pragma PIPELINE con intervalo de iniciación 1, de igual forma que para la función full connected.

```
#pragma HLS PIPELINE II= 1
```

- ```
  void conv (float out[], float in[], float weight[CONV_KERNEL_SIZE]
  [CONV_KERNEL_SIZE][CONV_CHANNELS][CONV_FILTERS],float bias[CONV_BIAS_SIZE])
  ```

De igual manera que en las anteriores funciones, para la función de convolución se aplican los mismos pragmas. En primer lugar, el pragma ARRAY_PARTITION tiene la misma justificación.

```
#pragma HLS ARRAY_PARTITION variable =in cyclic factor= 16 dim= 1
#pragma HLS ARRAY_PARTITION variable =out cyclic factor= 16 dim= 1
#pragma HLS ARRAY_PARTITION variable =weight cyclic factor= 32 dim= 4
#pragma HLS ARRAY_PARTITION variable =bias cyclic factor= 16 dim= 1
#pragma HLS ARRAY_PARTITION variable=conv_buff cyclic factor= 16 dim= 1
```

Los factores son elegidos según el tamaño de las variables que definen los tamaños de los arrays. Para el array weight[], al tener una dimensión de 4, se acceden a los pesos de los 32 filtros en paralelo (*CONV_FILTERS*).

Se desenrolla el bucle de los filtros parcialmente, procesando 4 filtros en paralelo.

```
#pragma HLS UNROLL factor= 4
```

Por último, se desenrolla completamente el bucle más interno (por canal). es eficiente ya que normalmente el número de canales es bajo (en este caso, *CONV_CHANNELS* = 1).

```
#pragma HLS UNROLL
```

- ```
  void top_cnn(float image[28 * 28], float fc_out[10], float conv_weights[3] [3] [1]
  [32], float conv_bias[32], float fc_weights[5408] [10], float fc_bias[10])
  ```

En la función principal, tan sólo aplicamos el pragma ARRAY_PARTITION a todos los parámetros de entrada. El factor y la dimensión dependerá de los arrays y sus tamaños.

Análisis de Resultados

Al realizar la síntesis C de Vitis, analizamos los resultados en la siguiente tabla:

	Latencia (ciclos)	Interval	Throughput	BRAM	DSP	FF	LUT
top_cnn	9018	9019	110.877	774	1590	263.016	264.989
conv	1633	1633	612.369	0	1454	131.243	137.664
pool	1356	1355	737.463	0	0	3225	17.498
fc	5881	5881	170.039	0	86	19.724	12.429

Como podemos observar, **se cumplen** las estimaciones del principio, disminuyendo mucho la latencia, y aumentando el uso de recursos (FF y LUT) puesto que se realizan más operaciones y se guardan más bits de información por ciclo de reloj, al tener pragmas de paralelización y desenrollado de los arrays.

En un ejemplo práctico, según datos recogidos en Internet, el clúster SIAM (Sistema Automático de Reconocimiento de Matrículas) [5] de la Guardia Civil Española trabaja a unos **20 y 100 milisegundos** en procesar una imagen, entre **60–120 fotogramas por segundo.** Es decir, el software tarda **50 ms** en detectar y reconocer la matrícula y la consulta en la base de datos tarda **10 ms** adicionales. Nuestro programa tiene una latencia de reloj de **0.108 ms** por operación de reconocimiento de dígitos, es decir tardaría unos 0.5 ms en detectar 4 números de una matrícula española. Por tanto, a priori, nuestra **OCR es más rápido que el sistema SIAM en la etapa de reconocimiento**, teniendo en cuenta que el SIAM no solo ejecuta OCR (incluye detección de vehículos, localización de la matrícula, preprocesamiento de la imagen y consulta en bases de datos) y que, la latencia del programa corresponde **solo al reconocimiento de dígitos**, no al sistema completo. A su vez, es lógico que **se utiliza hardware diferente**. El SIAM probablemente usa CPUs/GPUs genéricas, mientras que la FPGA está optimizada para una tarea específica. Por consecuente, ¿es óptimo comparar nuestro OCR en Vitis HLS con el clúster SIAM? **En la etapa de OCR puro** podría ser, **en el sistema completo** en teoría no, ya que existen muchos más factores que alteran las condiciones de velocidad de ejecución.

Conclusiones

Tras analizar las 3 arquitecturas y sus resultados, podemos llegar a una conclusión en cuanto a rendimiento-coste se supone, teniendo en cuenta cada caso de uso.

- ***¿Qué arquitectura es mejor?***

Para poder dar una respuesta exacta, lo mejor es comparar los resultados de todas las arquitecturas implementadas:

	Latencia (ciclos)	Interval	Throughput	BRAM	DSP	FF	LUT
ARQ. BASE	359.116	359.117	2808	**65**	**91**	28.585	50.322
ARQ. ITER.	**21.096.441**	**21.096.442**	**47**	**65**	**91**	**18.169**	**30.670**
ARQ. RÁPIDA	**9018**	**9019**	**110.877**	**774**	**1590**	**263.016**	**264.989**

Como podemos observar, la arquitectura base es la que ofrece un **equilibrio** entre velocidad y uso de recursos, la arquitectura iterativa por su parte tiene una **latencia muy alta** y un **throughput muy bajo**, a cambio de un **uso de unidades lógicas de memoria pequeño** y, la arquitectura rápida, ofrece una **latencia muy baja y un throughput alto**, pero un **uso de recursos muy alto**.

Así pues, para elegir una arquitectura frente a otra, es necesario **conocer el contexto** y el **uso** que se le vaya a dar al sistema. Si queremos detectar dígitos muy rápidamente sin tener restricciones de área o uso de memoria, la ***arquitectura rápida es la mejor opción***. Sin embargo, si por el contrario no nos importa la latencia o la velocidad del programa, pero sí el uso que se le dé a los recursos de memoria ya sea por restricción de la FPGA o coste de implementación, la ***arquitectura iterativa es la elegida***. Por otro lado, la arquitectura base (sin acelerar) estaría en un término medio, pero raramente se usará, o se busca rapidez, o se busca poco gasto de recursos.

- ***¿Qué uso es el recomendado para este sistema?***

En nuestra opinión, un sistema de reconocimiento de dígitos manuscritos utilizando redes neuronales, es más práctico es casos de baja latencia y alta velocidad, siendo estos tipos de sistemas utilizados por agentes de seguridad o de investigación, que trabajan a alta velocidad detectando muchos números en poco tiempo. Por ello, **elegimos la arquitectura rápida**, a pesar de un gasto de recursos de memoria alto, saldado por una FPGA que admita dicho inconveniente.

Anexo

¿Cómo funciona nuestro sistema de Reconocimiento de Dígitos con Redes Neuronales?

En este último apartado del informe, realizaremos una explicación del flujo de datos de nuestro programa, así como el funcionamiento del programa de testeo para imágenes de dígitos numéricos. Todos los archivos o librerías están incluidas en la carpeta Testbench.

Explicación del código

El código que ejecuta el testeo del programa es *main.cpp*. Este programa realiza pruebas automáticas sobre una red neuronal que llama a una función que lo implementa (*top_cnn.cpp*) dicha función está compuesta por 3 capas: **completamente conectado**, **convolucional** y **pool**, para ello se utiliza una base de datos de imágenes de dígitos almacenadas como archivos *.txt*. Cada archivo contiene una imagen representada como una lista de números flotantes (píxeles normalizados). El código recorre las carpetas 0 a 9, y evalúa el rendimiento del modelo clasificando las imágenes correspondientes a cada dígito.

Cuando se introduce una lista de números para saber a qué digito corresponde, los datos entran por unas capas estas están conectadas por medio de un enlace que hace conexión con el resto de las capas, por lo que, necesitamos que dicha conexión este bien implementa o lo que viene siendo "**entrenada**". Al ser el entrenamiento muy complicado, utilizamos librerías externas para ello (librerías de Python), y, este, nos da los sesgos y pesos para cada capa en formato de lista de números (array).

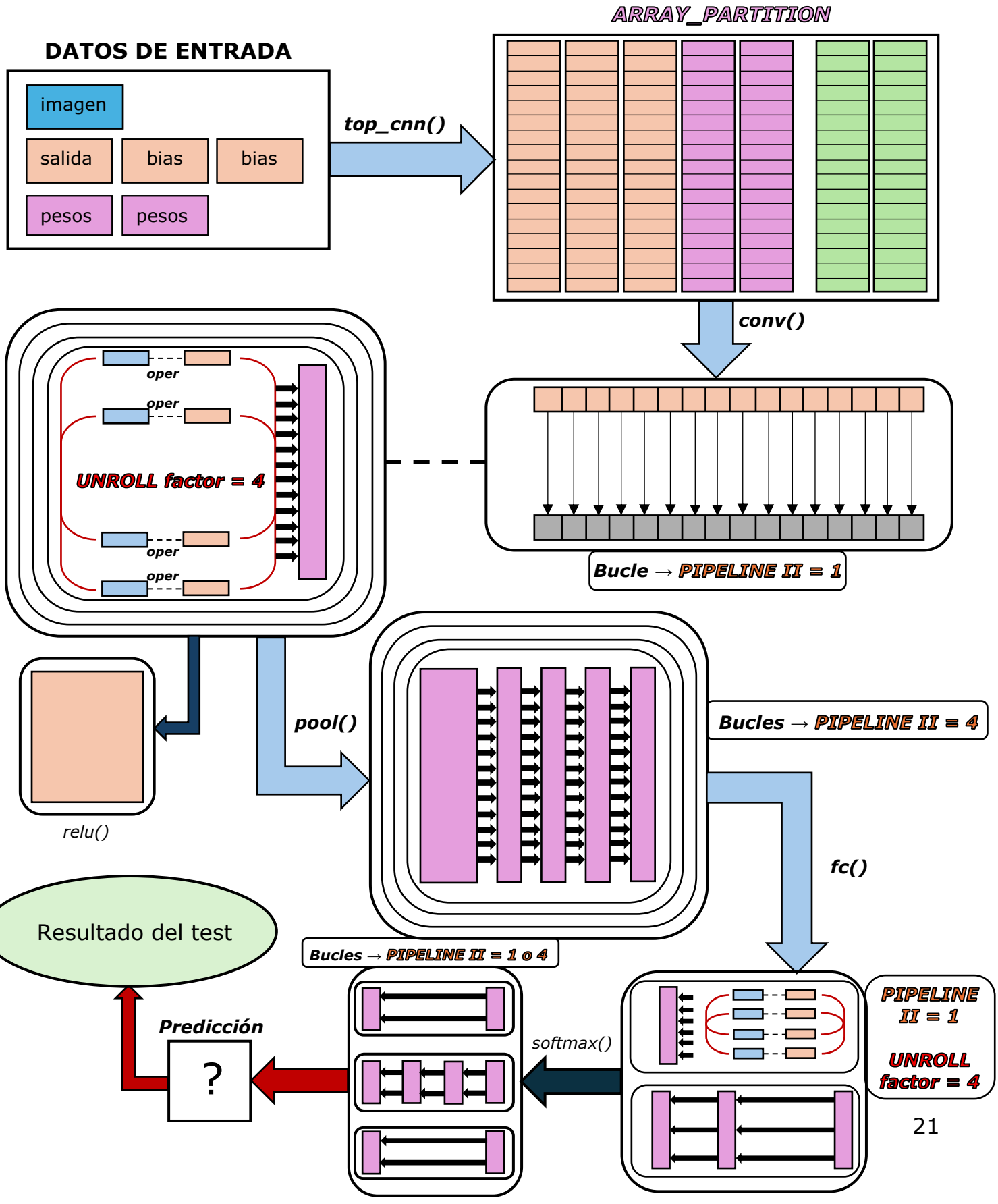
La estructura **testfile** contiene la ruta al archivo (path -> "*mnist_txt_100*")[6] y la etiqueta esperada (label -> "/dig1"), que corresponde al dígito (0-9).

La función ***collect_files*** realiza los siguientes pasos:

1. Recorre las carpetas base_dir/0, base_dir/1, ..., base_dir/9.
2. Dentro de cada una, busca archivos .txt.
3. Guarda la ruta del archivo y la etiqueta asociada en el array files.
4. Devuelve el número total de archivos encontrados.

Flujo de datos para la CNN acelerada

A continuación, se muestra un diagrama de flujo de datos en la que se puede ver el funcionamiento de la CNN acelerada, es decir, aplicándose pragmas de paralelización, desenrollado y partición de arrays.



Salida tras ejecutar el código de testeo

Para ejecutar nuestra CNN debemos tener **una o varias imágenes de dígitos numéricos**, las cuales el OCR se encargará de clasificar (gracias a la red neuronal entrenada) y devolvernos el **resultado**. Para ello, probamos con *10 carpetas* por dígito del *0 al 9* y *100 imágenes* de cada dígito, ya sea manuscrito de distintas formas o en otro formato de imagen, teniendo por tanto **1000 imágenes distintas**.

Nuestro archivo para ejecutar (*main.cpp*) se encargará de recoger los archivos de los subdirectorios. A continuación, lanzará la función de la CNN (pragmatizada o no) rellenando los parámetros de entrada y almacenando los valores de predicción, del cual selecciona el índice con el valor más alto, que corresponde a la clase predicha.

Después, compara la predicción con la etiqueta real: Si acierta, suma uno a *correct_predictions*. Si falla, imprime por pantalla el archivo mal clasificado, la predicción y la etiqueta real.

Por último, mostrará por consola los **resultados**, indicando el **número de imágenes testeadas**, las **predicciones** que han sido **correctas**, el **porcentaje** de **acierto** y el **tiempo** total de **ejecución**. Un ejemplo de ello sería:

```
Misclassified: mnist_txt_1000/1/31.txt -> predicted 3, true 1
Misclassified: mnist_txt_1000/2/87.txt -> predicted 9, true 2
Misclassified: mnist_txt_1000/3/50.txt -> predicted 2, true 3
Misclassified: mnist_txt_1000/5/6.txt -> predicted 1, true 5
Misclassified: mnist_txt_1000/6/43.txt -> predicted 0, true 6
Misclassified: mnist_txt_1000/7/21.txt -> predicted 1, true 7
Misclassified: mnist_txt_1000/7/15.txt -> predicted 9, true 7
Misclassified: mnist_txt_1000/8/32.txt -> predicted 5, true 8
Misclassified: mnist_txt_1000/9/82.txt -> predicted 2, true 9
Misclassified: mnist_txt_1000/9/9.txt -> predicted 8, true 9
Misclassified: mnist_txt_1000/9/93.txt -> predicted 0, true 9
Misclassified: mnist_txt_1000/9/19.txt -> predicted 7, true 9
###################### SUMMARY ######################
Total images tested: 1000
Correct predictions: 988
Accuracy: 98.8%
Total execution time: 11501.8 ms
```

Bibliografía

AMD. (s.f.). Obtenido de https://docs.amd.com/r/en-US/Vitis-Tutorials-Vitis-Hardware-Acceleration/Run-the-C-Synthesis

IBM. (s.f.). Obtenido de https://www.ibm.com/think/topics/machine-learning

Keras. (s.f.). Obtenido de https://keras.io/

Pytorch. (s.f.). Obtenido de https://pytorch.org/

SIAM. (s.f.). Obtenido de https://clustersiam.es/

TensorFlow. (s.f.). Obtenido de https://www.tensorflow.org/datasets/catalog/mnist?hl=es